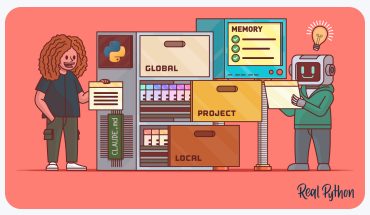

The CLAUDE.md Cheat Sheet

A learning resource for the Real Python "How to Write a CLAUDE.md File for Claude Code" tutorial.
Visit realpython.com to learn more.

The Three CLAUDE.md Files

GLOBAL

Personal Defaults

`~/.claude/CLAUDE.md`

Workflow rules and coding preferences that apply across all your projects.

- Working style
- Communication tone
- Python defaults
- Tool choices (`rg` , `uv`)

PROJECT

Repo Guidance (commit it!)

`./CLAUDE.md`

Repository-specific commands, conventions, and quirks. Shared with teammates.

- Install / test / lint commands
- Architecture constraints
- Non-obvious patterns
- Project-specific defaults

LOCAL

Private Notes (gitignored)

`./CLAUDE.local.md`

Machine-specific or personal overrides that shouldn't be committed.

- Local URLs and test data
- Personal preferences
- Temporary overrides
- Add to `.gitignore`

The Audit Workflow — Find What Claude Actually Needs

1 Hide Existing Files

Move global and project `CLAUDE.md` files aside. Verify with `/memory` .

2 Run a Real Task

Ask Claude Code to make a small, realistic change to your project.

3 Record Corrections

Note each time you have to redirect Claude. Repeated mistakes are gold.

4 Sort by Scope

Decide: global, project, or local? Write one targeted rule per correction.

Include — What Claude Can't Infer

- ✓ Build, test, lint, and format commands
- ✓ How to run a single test
- ✓ Environment setup that needs explanation
- ✓ Architecture boundaries and constraints
- ✓ Non-obvious patterns to avoid
- ✓ Project conventions not written down
- ✓ Pre-training quirks (e.g. AWS Bedrock global inference uses `global.` prefix)

Exclude — What Wastes Tokens

- × Long codebase summaries
- × Standard Python conventions Claude already knows
- × File-by-file directory descriptions
- × Documentation dumps
- × Anything Claude can read from the repo
- × Vague preferences ("make sure everything works")
- × Rules you've never had to give as a correction

Common Corrections → CLAUDE.md Rules

Used `pip install` imperatively	Use `uv add` for project deps; `uv pip install` only ad hoc	GLOBAL / PROJECT
Skipped the test command	Run `uv run pytest` before finishing a task	PROJECT
Added `print()` debugging	Use the configured `logging` module	PROJECT
Edited unrelated files	Touch only files needed for the requested change	GLOBAL
Opened broad files too early	Use `rg` to locate relevant files before opening source	GLOBAL

Global Starter — `~/.claude/CLAUDE.md`

```
# Global Claude Code Instructions

## Working Style
- State assumptions before editing.
- Prefer the smallest change that solves it.
- Touch only files needed for the change.
- Verify with the narrowest relevant test.

## Context and Tool Use
- Use rg -n <pat> <path> for exact matches.
- Review edits with git diff -- <path>.

## Python Defaults
- Use uv for deps and environments.
- Use logging, not print(), in app code.
- Add type hints to public functions.

## Communication
- Be concise. Call out trade-offs.
- Push back if a smaller approach works.
```

Tips & Troubleshooting

Verify Files Are Loaded

Run `/memory` inside Claude Code. You should see your global, project, and local files. If one is missing, check the file name and location.

Supported Locations

- `~/.claude/CLAUDE.md` — global
- `./CLAUDE.md` — project root
- `./.claude/CLAUDE.md` — project
- `./CLAUDE.local.md` — local

When Rules Are Ignored

- Shorten the file
- Remove conflicting instructions
- Replace vague preferences with concrete commands
- Tie each rule to a real correction

Import Syntax

Use `@AGENTS.md` or `@~/.claude/notes.md` to pull in another file. Note: imports still count toward context size.

The Golden Rules

1. Each rule fixes a **real, repeated** correction — not a hypothetical one.

2. Rules are **concrete commands**, not vague preferences ("run `uv run pytest` ", not "test it").

3. Keep it **short enough to skim**. Prune rules that no longer prevent mistakes.

4. Treat `CLAUDE.md` as **living context**, not static config. Revise as you go.